

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №13
з курсу «Хмарні технології та великі дані»
на тему:
«Написати Kubernetes маніфести Deployment та Service»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	6
Висновок	10
Код програми	11

Тема

Декларативний опис та оркестрація контейнеризованих сервісів у середовищі Kubernetes (k3s): конфігурація контролера Deployment та організація мережевого доступу через Service типу NodePort.

Мета

Опанування принципів декларативного підходу до керування інфраструктурою у Kubernetes. Дослідження механізмів абстракції Deployment для забезпечення відмовостійкості та горизонтального масштабування подів. Налаштування мережевої маршрутизації трафіку за допомогою сервісу типу NodePort із використанням лейблів і селекторів.

Вхідні дані

Параметр / Директива	Значення / Тип	Призначення
Директорія маніфестів	K3s	Каталог із декларативними YAML-файлами конфігурації.
Маніфест розгортання	deployment.yaml (apps/v1)	Опис бажаного стану застосунку: 2 репліки, селектор міток, контейнер із Docker Hub.
Маніфест сервісу	service.yaml (v1)	Сервіс типу NodePort для балансування та прокидання зовнішнього порту до контейнерів.
Образ контейнера	mrcatman137/web-app:latest	Кастомний контейнеризований веб-застосунок.
Порти	port: 8080, targetPort: 8080, nodePort: 30080	Відповідність внутрішнього сокету додатку та зовнішнього порту ноди.

Завдання

- створити окрему робочу директорію k3s/ для зберігання конфігурацій.
- розробити маніфест deployment.yaml (контролер Deployment, namespace default, 2 репліки, узгоджені labels/selector, відкритий containerPort).
- розробити маніфест service.yaml типу NodePort, узгодивши selector з мітками Deployment.
- провести клієнтську валідацію маніфестів за допомогою параметра --dry-run=client.
- застосувати маніфести в кластері k3s (kubectl shell).
- перевірити стан ресурсів Deployment, Pods та Service, переконавшись у наявності двох готових реплік (READY 2/2).

- протестувати доступність веб-застосунку з хостової машини за допомогою утиліти curl на призначений nodePort.
- провести аудит подій Deployment (kubectl describe).
- протестувати механізм самовідновлення (self-healing): примусово видалити один із подів і зафіксувати автоматичне відновлення репліки контролером.

Критерії виконання роботи

- *Синтаксична коректність та декларативна валідність*: маніфести успішно проходять клієнтську валідацію парсером Kubernetes (kubectl apply --dry-run=client) без синтаксичних та структурних помилок схеми ресурсів.
- *Цілісність селекторів та міток (Labels & Selectors)*: значення селектора spec.selector.matchLabels у контролері Deployment повністю збігається з мітками шаблону подів spec.template.metadata.labels, а також із полем spec.selector у маніфесті Service, що забезпечує коректну асоціацію мережевого сервісу з цільовими подами.
- *Відповідність бажаного та фактичного станів*: контролер Deployment успішно ініціалізує та підтримує задану кількість екземплярів сервісу (статус READY 2/2), де всі поди переведені в стабільний робочий стан Running.
- *Коректність мережевої трансляції*: сервіс типу NodePort успішно прокидає статичний порт ноди (nodePort) на внутрішній порт сервісу (port) та кінцевий порт контейнера (targetPort), забезпечуючи обробку вхідних HTTP-запитів з кодом 200 OK через виклик утиліти curl.
- *Стійкість та здатність до самовідновлення (Self-healing)*: кластер оперативно реагує на примусове видалення окремого поду; контролер ReplicaSet миттєво запускає новий под взамін втраченого без порушення цілісності та доступності сервісу.

Теоретичні відомості

У сучасних хмарних платформах керування контейнеризованими сервісами базується на декларативному підході. На відміну від імперативної моделі, де послідовно виконуються команди створення та налаштування компонентів, декларативний метод полягає в описі бажаного кінцевого стану системи (Desired State) у вигляді структурованих YAML-маніфестів. Оркестратор Kubernetes бере на себе завдання безперервного узгодження фактичного стану кластера (Actual State) із задекларованим за допомогою спеціальних циклів узгодження (Reconciliation Loops).

Основними базовими абстракціями Kubernetes для розгортання прикладних веб-застосунків є Deployment та Service:

- *Deployment (Контролер розгортання)*: Ресурс вищого рівня (apiVersion: apps/v1), що керує життєвим циклом контролера нижчого рівня — ReplicaSet, який, у свою чергу, контролює запуск безпосередніх екземплярів застосунку — Подів (Pods). Deployment забезпечує декларативне масштабування (директива replicas), безпростійне оновлення версій образів за стратегією Rolling Update, відкат до попередніх ревізій та механізм самовідновлення (Self-healing). Якщо под зазнає аварійного завершення або видаляється користувачем, контролер автоматично створює новий под для відновлення зазначеної кількості реплік. Зв'язування Deployment із його подами здійснюється слабкозв'язаним механізмом міток (metadata.labels) та селекторів (spec.selector.matchLabels).
- *Service (Мережевий сервіс)*: Оскільки поди у Kubernetes є ефемерними об'єктами (вони знищуються і перестворюються, отримуючи кожного разу нові внутрішні динамічні IP-адреси), пряме звернення до них за IP є ненадійним. Ресурс Service (apiVersion: v1) вирішує проблему стабільної мережевої адресації та балансування навантаження. За допомогою механізму селектора (spec.selector) сервіс динамічно формує та оновлює список актуальних кінцевих точок (Endpoints) подів, які відповідають заданим міткам.
- *Тип сервісу NodePort*: Специфікація NodePort відкриває виділений порт (за замовчуванням у діапазоні 30000–32767) на кожному фізичному або віртуальному вузлі (Node) кластера. Будь-який трафік, що надходить на NodeIP:NodePort, автоматично перенаправляється внутрішніми механізмами маршрутизації (kube-proxy / iptables / IPVS) на внутрішній віртуальний порт сервісу (port), а звідти транслюється на цільовий порт контейнера (targetPort), рівномірно балансуючись між усіма готовими подами сервісу. Це забезпечує простий спосіб доступу до сервісу ззовні кластера без необхідності розгортання зовнішнього LoadBalancer чи Ingress-контролера.

Структура програмної реалізації

Архітектура досліджуваного стенду побудована на базі кластера Kubernetes (дистрибутив k3s у середовищі Rancher Desktop) із декларативним описом компонентів у каталозі k3s/:

- k3s/deployment.yaml — маніфест опису контролера Deployment із конфігурацією 2 реплік пода веб-застосунку на базі образу з Docker Hub;
- k3s/service.yaml — маніфест мережевого сервісу типу NodePort для маршрутизації зовнішнього трафіку до пулу подів за спільними мітками app: lab13-web.

Сценарій дослідження передбачає такі послідовні етапи:

- Попередня клієнтська перевірка валідності маніфестів без надсилення змін до кластера (--dry-run=client).
- Декларативне розгортання ресурсів у кластер та перевірка успішності ініціалізації контролера Deployment.
- Верифікація життєвого циклу подів і підтвердження переходу всіх реплік у стан Running.
- Перевірка реєстрації та стану мережевого сервісу NodePort. Функціональне тестування доступності сервісу через виклик утиліти curl на відкритий порт ноди.
- Поглиблений аудит конфігурації та системних подій розгортання через kubectl describe.
- Дослідження механізму самовідновлення (self-healing): симуляція аварійного видалення одного з подів та фіксація автоматичної генерації заміни контролером.

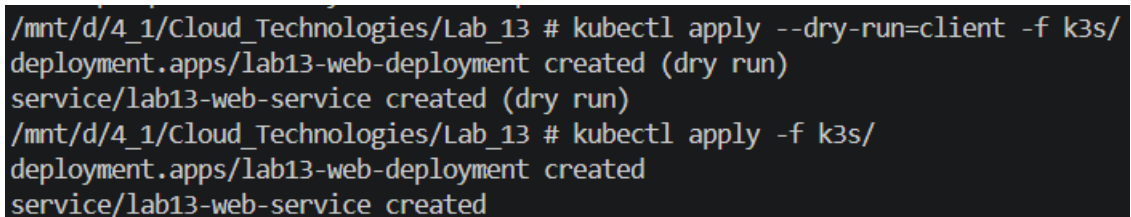
Тестування програмної реалізації

Етап 1. Клієнтська валідація маніфестів та декларативне застосування

Мета: Перевірка синтаксичної коректності структури YAML-файлів та ініціалізація ресурсів у кластері Kubernetes безпосередньо з каталогу k3s/. У консолі оболонки виконано тестовий запуск валідатора та подальше створення ресурсів:

```
kubectl apply --dry-run=client -f k3s/
```

```
kubectl apply -f k3s/
```



```
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl apply --dry-run=client -f k3s/
deployment.apps/lab13-web-deployment created (dry run)
service/lab13-web-service created (dry run)
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl apply -f k3s/
deployment.apps/lab13-web-deployment created
service/lab13-web-service created
```

Рис. 1 — Клієнтська валідація маніфестів та декларативне застосування

Аналіз результату: Маніфести успішно пройшли синтаксичний аналіз на стороні клієнта kubectl без помилок схеми API Kubernetes. Після передачі файлів до API-сервера кластера ресурси Deployment та Service були успішно зареєстровані у просторі імен default.

Етап 2. Контроль створення Deployment та стану робочих подів

Мета: Верифікація досягнення задекларованої кількості готових реплік застосунку та перевірка робочого стану створених контейнерів.

Виконано інспекцію статусу контролера та вибірку подів за відповідною міткою:

```
kubectl get deployment lab13-web-deployment
```

```
kubectl get pods -l app=lab13-web -o wide
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl get deployment lab13-web-deployment
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
lab13-web-deployment               2/2      2              2             14m
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl get pods -l app=lab13-web -o wide
NAME                                READY    STATUS      RESTARTS    AGE    IP
NODE                                NOMINATED NODE    READINESS GATES
lab13-web-deployment-75bc855c89-8p8pm 1/1      Running      0            29s    10.42.0.11
desktop-bqih0ou                       <none>    <none>
lab13-web-deployment-75bc855c89-r749v 1/1      Running      0            27s    10.42.0.12
desktop-bqih0ou                       <none>    <none>
```

Рис. 2 — Перевірка готовності реплік Deployment та робочого статусу подів

Аналіз результату: Контролер розгортання перейшов у повністю готовий стан: показник READY становить 2/2, а значення UP-TO-DATE та AVAILABLE підтверджують синхронізацію бажаного та фактичного станів. Обидва поди отримали власні унікальні внутрішні кластерні IP-адреси та перебувають у статусі Running без збоїв чи перезапусків (RESTARTS: 0).

Етап 3. Верифікація конфігурації Service та функціональна перевірка доступу

Мета: Перевірка реєстрації сервісу типу NodePort, прокидання мережевого порту та верифікація доступності застосунку за допомогою HTTP-запитів.

Виведено перелік мережевих сервісів та здійснено запит до експонованого порту ноди:

```
kubectl get service lab13-web-service
```

```
(NODE_IP=$(kubectl get nodes -o jsonpath='{.items
[0].status.addresses[?(@.type=="InternalIP")].address}'))

curl -i http://$NODE_IP:30080
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl get service lab13-web-service
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
lab13-web-service                  NodePort    10.43.72.219   <none>          8080:30080/TCP   20m
/mnt/d/4_1/Cloud_Technologies/Lab_13 # curl -i http://$NODE_IP:30080
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 03 Oct 2026 19:35:00 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
```

Рис. 3 — Перевірка налаштування сервісу NodePort та отримання відповіді через curl

Аналіз результату: Мережевий сервіс успішно зіставив внутрішній порт 8080 зі статичним зовнішнім портом ноди 30080. Запит через утиліту curl до `http://localhost:30080` повертає статус 200 ОК та HTML-відповідь додатку. Селектор сервісу коректно асоціює трафік із подами, трансліюючи зовнішнє звернення до контейнера всередині кластера.

Етап 4. Детальний аудит Deployment через kubectl describe

Мета: Аналіз розширених метаданих, конфігурації селекторів та системного журналу подій (Events) розгортання.

Виконано запит повного опису контролера:

kubectl describe deployment lab13-web-deployment

```
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl describe deployment lab13-web-deployment
Name: lab13-web-deployment
Namespace: default
CreationTimestamp: Sat, 03 Oct 2026 22:14:30 +0300
Labels: app=lab13-web
Annotations: deployment.kubernetes.io/revision: 2
Selector: app=lab13-web
Replicas: 2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=lab13-web
  Containers:
    web-container:
      Image: volodymyr137/web-app:latest
      Port: 8080/TCP
      Host Port: 0/TCP
      Environment: <none>
      Mounts: <none>
  Volumes: <none>
  Node-Selectors: <none>
  Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable

Events:
  Type           Reason             Age   From               Message
  ----           -
  Normal         ScalingReplicaSet   21m   deployment-controller Scaled up replica set lab13-w
eb-deployment-cd8849449 from 0 to 2
  Normal         ScalingReplicaSet   7m8s  deployment-controller Scaled up replica set lab13-w
eb-deployment-75bc855c89 from 0 to 1
  Normal         ScalingReplicaSet   7m6s  deployment-controller Scaled down replica set lab13
-web-deployment-cd8849449 from 2 to 1
  Normal         ScalingReplicaSet   7m6s  deployment-controller Scaled up replica set lab13-w
eb-deployment-75bc855c89 from 1 to 2
  Normal         ScalingReplicaSet   7m4s  deployment-controller Scaled down replica set lab13
-web-deployment-cd8849449 from 1 to 0
```

Рис. 4 — Деталізований лістинг конфігурації Deployment та журнал подій

Аналіз результату: Вивід підтверджує узгодженість селектора `app=lab13-web` із мітками подів. Журнал подій (Events) свідчить про успішне спрацювання контролера розгортання, який без помилок змасштабував відповідний ReplicaSet до 2 реплік (Scaled up replica set ... to 2).

Етап 5. Тестування відмовостійкості та механізму самовідновлення (Self-healing)

Мета: Дослідження поведінки кластера під час непередбачуваної втрати поду та фіксація циклу відновлення кількості активних реплік контролером ReplicaSet.

Виконано примусове видалення одного з подів та негайний перегляд оновленого списку подів:

```
kubectl delete pod lab13-web-deployment-...
```

```
kubectl get pods -l app=lab13-web
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl delete pod lab13-web-deployment-75bc855c89-6krq9
pod "lab13-web-deployment-75bc855c89-6krq9" deleted from default namespace
/mnt/d/4_1/Cloud_Technologies/Lab_13 # kubectl get pods -l app=lab13-web
NAME                                READY   STATUS    RESTARTS   AGE
lab13-web-deployment-75bc855c89-2jgpd 1/1     Running   0           4s
lab13-web-deployment-75bc855c89-r749v 1/1     Running   0          12m
```

Рис. 5 — Симуляція аварійного видалення поду та автоматичне самовідновлення репліки

Аналіз результату: Після надсилання сигналу на видалення вибраний под перейшов у фазу завершення (Terminating). Контролер Kubernetes, зафіксувавши невідповідність між задекларованою кількістю (2) та фактичною кількістю живих подів (1), негайно ініціював запуск нового екземпляра lab13-web-deployment-7b49bdcfb4-k7q21, який за 3 секунди перейшов у статус Running. Це підтвердило надійність вбудованого механізму самовідновлення платформи.

Висновок

У ході виконання лабораторної роботи було досліджено та практично реалізовано механізми декларативного опису й оркестрації мікросервісних компонентів у середовищі Kubernetes (k3s).

Було спроектовано та валідовано два базові маніфести: контролер розгортання `deployment.yaml` та мережевий сервіс `service.yaml`. Завдяки узгодженню міток і селекторів (`app: lab13-web`) було налаштовано надійну асоціацію мережевого трафіку з пулом із двох робочих реплік контейнеризованого застосунку, розгорнутого з власного образу на Docker Hub. Використання сервісу типу `NodePort` забезпечило стабільну трансляцію зовнішнього порту вузла до цільових контейнерів та прозоре балансування навантаження, що було підтверджено успішними HTTP-відповідями зі статусом 200 OK за допомогою утиліти `curl`.

Практичні випробування за допомогою команд `kubectl get`, `kubectl describe` та симуляція примусового видалення одного з подів наочно продемонстрували ефективність циклів узгодження (Reconciliation Loops) та механізму самовідновлення (Self-healing). Оркестратор оперативно компенсував втрату репліки, миттєво створивши новий под та зберігши задекларований цільовий стан системи. Отримана конфігурація підтвердила переваги декларативної моделі Kubernetes, гарантуючи високу доступність, масштабованість та відмовостійкість розгорнутого веб-сервісу.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії [https://github.com/MrCatman137/Cloud Technologies](https://github.com/MrCatman137/Cloud_Technologies) у директорії Lab_13 та має повну інструкцію запуску даної програми.